Python Engineering - Data Growth Guide

Listas, preallocation y np.vstack(): como elegir segun crecen los datos

Objetivo de esta guia

Esta guia sirve para decidir como construir trayectorias, historiales de estados, resultados de integracion o datasets que van creciendo durante una simulacion. La pregunta principal es:

Conozco de antemano cuantas filas voy a necesitar?

Esa pregunta suele decidir entre tres estrategias: `list.append()`, preallocation de NumPy y `np.vstack()`.

1. Antes de elegir: que significa que los datos crecen?

Imagina que en cada paso de una simulacion guardamos un estado [h, v, t]. Cada iteracion produce una nueva fila.

```
step 0 -> [h0, v0, t0]
step 1 -> [h1, v1, t1]
step 2 -> [h2, v2, t2]
...

final structure:
[[h0, v0, t0],
 [h1, v1, t1],
 [h2, v2, t2],
 ...]
```

El resultado final es naturalmente una matriz NumPy de shape `(N, 3)`. La cuestion es como construirla durante el loop.

2. Estrategia A - lista + append()

Usamos una lista Python como contenedor dinamico mientras los datos crecen. Cuando termina la generacion, convertimos una sola vez a NumPy.

```
rows = []

for ...:
    new_state = np.array([h, v, t])
    rows.append(new_state)

trajectory = np.array(rows)
```

Cuando usarla: cuando no conocemos N de antemano o la simulacion puede terminar por una condicion.

Ejemplos: integrar hasta impacto, leer un archivo de longitud desconocida, detectar eventos, filtrar datos con un numero de resultados impredecible.

3. Por que las listas crecen bien?

Una lista de Python esta disenada para crecer dinamicamente. `append()` no obliga normalmente a reconstruir toda la lista en cada iteracion. Python reserva capacidad adicional internamente y la amplia cuando es necesario.

No necesitas memorizar los detalles de memoria. La idea util es: **si el numero de elementos cambia de forma impredecible, una lista es un buen contenedor de construccion.**

4. Estrategia B - preallocation NumPy

Si conocemos el numero final de filas y columnas, podemos reservar toda la matriz antes del loop y rellenarla directamente.

```
n_steps = 1000
trajectory = np.empty((n_steps + 1, 3))

trajectory[0] = initial_state

for i in range(n_steps):
    # propagate state
    trajectory[i + 1] = new_state
```

Cuando usarla: cuando N es conocido y la estructura final tiene forma fija.

Esto suele ser una opcion excelente en metodos numericos: Euler con un numero fijo de pasos, Runge-Kutta sobre una malla conocida, simulaciones con tiempo inicial/final y dt definidos.

5. Por que preallocation suele ser la mas eficiente?

NumPy puede reservar una sola vez el bloque de memoria necesario para toda la matriz. Despues solo escribimos en posiciones ya existentes.

```
reserve once:
(1001, 3)

then fill:
row 0
row 1
row 2
...
```

No necesitamos crear una matriz nueva en cada iteracion. Esto reduce copias y suele mejorar rendimiento cuando N es grande.

6. np.empty() vs np.zeros()

Funcion	Inicializacion	Uso tipico
np.empty(shape)	Memoria sin inicializar con valores definidos	Vamos a sobrescribir todas las posiciones
np.zeros(shape)	Todo empieza en 0	El cero tiene significado o queremos inicializacion segura

`np.empty()` puede contener valores residuales hasta que los sobrescribimos. No debemos leer posiciones que aun no hemos rellenado.

7. Estrategia C - np.vstack()

`np.vstack()` une arrays verticalmente: coloca nuevas filas debajo de las existentes.

```
trajectory = np.array([[h0, v0, t0]])

for ...:
    new_state = np.array([h, v, t])
    trajectory = np.vstack((trajectory, new_state))
```

El codigo es valido y muy legible para ejemplos pequenos.

8. Que ocurre realmente con vstack dentro de un loop?

Los arrays NumPy tienen tamano fijo. Cuando hacemos un vstack que aumenta el numero de filas, NumPy necesita producir un array nuevo con la forma mayor.

```
iteration 1:
(1,3) -> create (2,3) -> copy old data -> add new row

iteration 2:
(2,3) -> create (3,3) -> copy old data -> add new row

iteration 3:
(3,3) -> create (4,3) -> copy old data -> add new row
```

Con pocas filas no importa. Con miles o millones de iteraciones, repetir esas copias puede resultar mucho menos eficiente que preallocation o list.append().

9. Entonces, np.vstack() es malo?

No. **vstack es muy util.** El problema no es la funcion; el problema es usarla repetidamente para hacer crecer un array una fila cada vez dentro de un loop grande.

Buenos usos de vstack:

```
# combine a few complete blocks
all_data = np.vstack((experiment_A, experiment_B))

# join arrays after processing
combined = np.vstack((upper_block, lower_block))

# small educational loop
trajectory = np.vstack((trajectory, new_state))
```

Regla practica: **vstack es excelente para unir arrays; no suele ser la primera opcion para implementar una estructura dinamica de gran tamano.**

10. Comparacion directa

Situacion	Mejor opcion habitual	Motivo
N desconocido	list + append	Crecimiento dinamico natural
N conocido	NumPy preallocation	Reserva una vez y rellena
Unir pocos arrays completos	np.vstack	Claro y directo
Loop pequeno didactico	np.vstack puede valer	Simplicidad > micro-optimizacion
Loop grande + N conocido	Preallocation	Evita reallocations/copias
Loop grande + N desconocido	List + append -> np.array	Evita reconstruir arrays

11. Ejemplo de ingenieria: integrar hasta impacto

No sabemos exactamente cuantas iteraciones necesitaremos porque la simulacion termina cuando h <= 0.

```
rows = []

while h > 0.0:
    rows.append([h, v, t])
    # Euler propagation

trajectory = np.array(rows)
```

Eleccion: lista dinamica.

12. Ejemplo de ingenieria: 60 s con dt=0.01 s

Aqui conocemos el tiempo final y dt. Podemos calcular el numero de pasos antes de empezar.

```
t0 = 0.0
tf = 60.0
dt = 0.01

n_steps = int((tf - t0) / dt)
trajectory = np.empty((n_steps + 1, 3))

for i in range(n_steps):
    trajectory[i + 1] = new_state
```

Eleccion: preallocation NumPy.

13. Ejemplo de ingenieria: combinar dos campañas

```
campaign_1.shape
# (500, 6)

campaign_2.shape
# (300, 6)

all_measurements = np.vstack((campaign_1, campaign_2))
# shape -> (800, 6)
```

Eleccion: vstack, porque estamos uniendo bloques completos, no creciendo fila a fila.

14. Cuidado con shapes al usar vstack

```
state = np.array([100.0, 20.0, 0.0])
state.shape
# (3,)

trajectory = np.array([state])
trajectory.shape
# (1, 3)
```

Para una trayectoria conviene pensar desde el principio en **filas = estados** y **columnas = variables**.

15. Listas y NumPy no son rivales

Es frecuente usar una lista durante la fase de construccion y NumPy durante la fase de calculo. Cada estructura tiene fortalezas diferentes.

```
dynamic construction
       |
       v
Python list
       |
       | np.array(...)
       v
NumPy array
       |
       v
vectorized analysis
```

16. Decision tree

```
DO I KNOW THE FINAL NUMBER OF ROWS?
            |
      +-----+-----+
      |           |
     YES          NO
      |           |
      v           v
PREALLOCATE     LIST + APPEND
NUMPY           then np.array()
      |
      v
fill by index

SEPARATE QUESTION:
Do I only need to combine a few existing arrays?
      |
     YES
      |
      v
np.vstack() is a good fit
```

17. Regla mental para metodos numericos

> Cuando lleguemos a integradores numericos, piensa primero en la malla temporal:

```
fixed time grid?
    -> usually know N
    -> preallocate

event-driven / unknown stopping time?
    -> unknown N
    -> list + append

joining completed result blocks?
    -> vstack
```

18. Errores tipicos

Error	Problema	Mejor idea
vstack en miles de iteraciones	Recrea/copia arrays repetidamente	Preallocate o lista
np.empty y leer antes de rellenar	Valores residuales	Rellenar todo antes de usar
lista y esperar operaciones vectorizadas	No es NumPy	Convertir a np.array al final
empezar trayectoria como shape (3,)	Forma cambia al primer vstack	Empezar como (1,3)
preallocate sin conocer realmente N	Puede sobrar/faltar espacio	Lista dinamica

19. Cheat sheet final

```
UNKNOWN N
rows = []
rows.append(new_row)
A = np.array(rows)

KNOWN N
A = np.empty((N, n_columns))
A[i] = new_row

JOIN EXISTING ARRAYS
A = np.vstack((A1, A2))

KEY IDEA
list      -> dynamic growth
NumPy     -> fixed numerical structure
vstack    -> joining arrays
prealloc  -> efficient known-size simulation
```

No busques una herramienta universal. La buena eleccion depende de si conoces el tamano final, si los datos crecen dinamicamente y si estas construyendo o simplemente combinando arrays.